

Advanced Programming

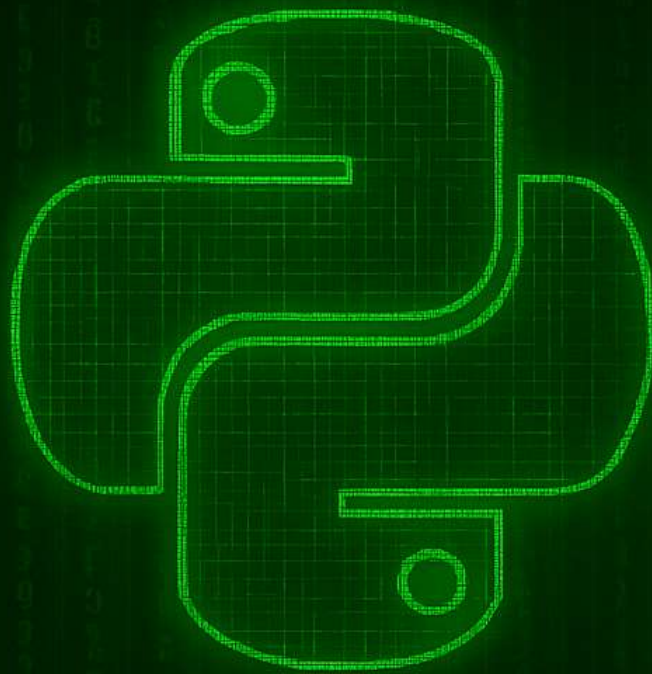
**Python repetition and advanced
Python**



Today's Topics

- General course information
- Semester plan
- Grading / Semester project

- Content:
 - Brief Python Foundations recap
 - Type Hints
 - Enumerators
 - Generators
 - Procedure models and agile methods
- Time for exercises and group formation/project brainstorming



PYTHON PROGRAMMING COURSE RECAP

Variables and Data Types

Common Data Types

- **int**: Whole numbers (42, -10)
- **float**: Decimal numbers (3.14, -0.5)
- **str**: Text ("Hello", 'World')
- **bool**: True or False

Examples

```
# Variable assignment
age = 25
price = 19.99
name = "Alice"
is_student = True

# Check type
type(age) # <class 'int'>
```

Boolean Logic & Comparison

Comparison Operators

```
5 == 5      # True (equal)
5 != 3      # True (not equal)
5 > 3       # True
5 < 10      # True
5 >= 5      # True
5 <= 3      # False
```

Logical Operators

```
age = 25
has_ticket = True

# AND - both must be True
if age >= 18 and has_ticket:
    print("Can enter")

# OR - at least one True
if age < 5 or age > 65:
    print("Discount")

# NOT - inverts
if not has_ticket:
    print("Buy one")
```

While Loops

Basic While Loop

```
# Count from 1 to 5
count = 1
while count <= 5:
    print(count)
    count += 1
# Output: 1 2 3 4 5
```

Loop Control

```
# break - exit loop
while True:
    answer = input("Quit? ")
    if answer == "yes":
        break

# continue - skip iteration
n = 0
while n < 5:
    n += 1
    if n == 3:
        continue # Skip 3
    print(n) # 1 2 4 5
```

For Loops

Using range()

```
# range(stop)
for i in range(5):
    print(i)    # 0 1 2 3 4

# range(start, stop)
for i in range(1, 6):
    print(i)    # 1 2 3 4 5

# range(start, stop, step)
for i in range(0, 10, 2):
    print(i)    # 0 2 4 6 8
```

Iterating Over Lists

```
fruits = ["apple", "banana"]

# Direct iteration
for fruit in fruits:
    print(fruit)

# With index
for i in range(len(fruits)):
    print(i, fruits[i])
```

Functions: Basics

Defining and Calling Functions

```
# Define a function
def greet():
    print("Hello!")

# Call the function
greet()  # Output: Hello!
```

Why Use Functions?

- **Reusability:** Write once, use many times
- **Organization:** Break complex programs into smaller pieces
- **Maintainability:** Easier to update and debug

Parameters & Return Values

Parameters

```
# Function with parameters
def greet(name, age):
    print(f"Hi {name}!")
    print(f"Age: {age}")

# Call with arguments
greet("Alice", 25)

# Default parameters
def power(base, exp=2):
    return base ** exp
```

Return Values

```
# Return a value
def add(a, b):
    return a + b

result = add(5, 3)
print(result) # 8

# Return multiple values
def stats(nums):
    return min(nums), max(nums)

low, high = stats([1, 5, 3])
```

Reading Files

Reading Entire File

```
# Read all at once
file = open("data.txt", "r")
content = file.read()
file.close()

# Better: with statement
with open("data.txt", "r") as f:
    content = f.read()
# File auto-closes!
```

Reading Line by Line

```
# Read all lines
with open("data.txt") as f:
    lines = f.readlines()

# Loop through lines
with open("data.txt") as f:
    for line in f:
        print(line.strip())
```

Writing Files

Writing Text

```
# Write (overwrites file)
with open("out.txt", "w") as f:
    f.write("Hello\n")
    f.write("World\n")

# Append (adds to end)
with open("out.txt", "a") as f:
    f.write("New line\n")
```

Writing Multiple Lines

```
lines = ["Line 1\n",
         "Line 2\n",
         "Line 3\n"]

with open("data.txt", "w") as f:
    f.writelines(lines)
```

Exception Handling: try/except

Basic Exception Handling

```
try:
    num = int(input("Number: "))
    result = 10 / num
    print(result)
except ValueError:
    print("Invalid number!")
except ZeroDivisionError:
    print("Can't divide by 0!")
```

Catch All Exceptions

```
try:
    # risky code
    file = open("data.txt")
except FileNotFoundError:
    print("File not found!")
except Exception as e:
    print(f"Error: {e}")
```

try/except/else/finally

Complete Structure

```
try:
    num = int(input("Enter: "))
    result = 10 / num
except ValueError:
    print("Invalid input")
except ZeroDivisionError:
    print("Cannot divide by 0")
else:
    print(f"Result: {result}")
finally:
    print("Done!")
```

When Each Executes

try: Always runs first

except: Runs ONLY if error occurs in try block

else: Runs ONLY if NO error occurred (try succeeded)

finally: ALWAYS runs, regardless of errors

Use **finally** for cleanup (closing files, connections)

Lists: Basics

Creating and Accessing

```
# Create a list
fruits = ["apple", "banana"]
numbers = [1, 2, 3, 4, 5]

# Access by index (0-based)
print(fruits[0]) # apple
print(fruits[-1]) # banana

# Length
len(fruits) # 2
```

Modifying Lists

```
nums = [1, 2, 3]

# Add items
nums.append(4) # [1, 2, 3, 4]
nums.insert(0, 0) # [0, 1, 2, 3, 4]

# Remove items
nums.remove(2) # [0, 1, 3, 4]
nums.pop() # [0, 1, 3]
```

String Methods

Case and Formatting

```
text = "Hello World"

text.upper()    # HELLO WORLD
text.lower()    # hello world
text.title()    # Hello World

" text ".strip() # text
text.replace("World", "Python")
```

Searching and Splitting

```
text = "Python is fun"

# Find substring
text.find("is")      # 7
"is" in text         # True

# Split into list
words = text.split()
# ['Python', 'is', 'fun']
```

String Formatting

Concatenation

```
name = "Alice"
age = 25

# Using + operator
msg = "Hi " + name + "!"

# Using comma in print
print("Age:", age)
```

F-strings (Modern)

```
name = "Bob"
age = 30
price = 19.99

# Embed variables
msg = f"Hi {name}!"
info = f"{name} is {age}"

# Format numbers
f"Price: ${price:.2f}"

# Output: Price: $19.99
```


Lists – Index

- Index: number specifying the position of an element in a list

```
some_list = ['a', 2, 'hi there!', 42]
```

Index: 0 1 2 3



- Indexing starts at 0
- Index of last element: List length -1 or just -1. In this example: `some_list[3] = 42` and `some_list[-1] = 42`
- Negative index: position relative to the end of the list:
 - index -1: last element
 - index -2: next to last element
 - etc.

Lists – Indexing

– Access elements:

```
>>> some_list = ['a', 2, 'hi there!', 42]
```

```
>>> some_list[1]  
2
```

```
>>> some_list[2:3]  
['hi there!']
```

```
>>> some_list[2:4]  
['hi there!', 42]
```

```
>>> some_list[0:4:2]  
['a', 'hi there!']
```

step is not mandatory

General format:

`some_list[start:stop:step]`

Two-Dimensional Lists

Figure 7-5 A two-dimensional list

	Column 0	Column 1
Row 0	'Joe'	'Kim'
Row 1	'Sam'	'Sue'
Row 2	'Kelly'	'Chris'

```
>>> two_dim_list = [['Joe', 'Kim'], ['Sam', 'Sue'], ['Kelly', 'Chris']]

>>> len(two_dim_list)
3

>>> two_dim_list[1]
['Sam', 'Sue']

>>> two_dim_list[1][1]
'Sue'
```

List Comprehensions

- List comprehension: a concise expression that creates a new list by iterating over the elements of an existing list.

`list2 = [item for item in list1]`



Result Expression Iteration Expression

```
list1 = [1, 2, 3, 4]
list2 = [item**2 for item in list1]
```

List Comprehensions with conditions

```
numbers = [1, 100, 20, 5, 42]
```

```
numbers_filtered = [x for x in oldlist if x < 5]
```

```
fruits = ['apples', 'banana', 'kiwi', 'strawberry', 'pears', 'mango', 'pineapple']
```

```
newfruits = [x if x != "banana" else "orange" for x in fruits]
```

In case you need if and else, the structure changes

Tuples

- Tuple: an immutable sequence
 - a. Similar to a list
 - b. Tuples are immutable. Once a tuple is created:
 - New elements cannot be added to the tuple
 - Elements cannot be removed from the tuple
 - The values stored in tuple elements cannot be changed

Tuples

- Creating a Tuple

```
a.my_tuple= (item1, item2)
```

```
b.my_tuple= (item1,)
```

- Tuples support many list operations

- Subscript indexing for retrieving elements
- Methods such as `index`
- Built in functions such as `len`, `min`, `max`
- Slicing expressions
- The `in`, `+`, and `*` operators

Tuples

- Since tuples are immutable, they **do not support the methods**:

`a.append`

`b.remove`

`c.insert`

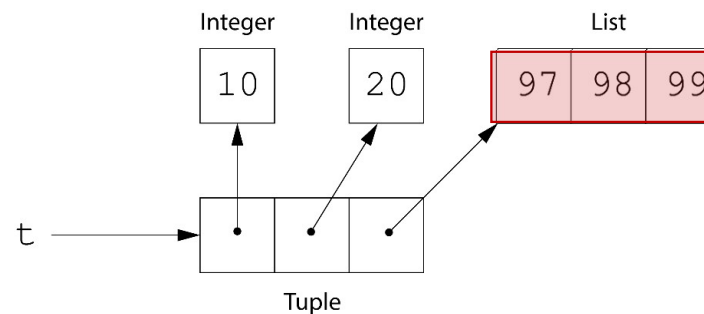
`d.reverse`

`e.sort`

Storing Mutable Objects in a Tuple

- Although tuples are immutable, you can store mutable objects in a tuple.
 - For example, the following statement creates a tuple that contains a list as its 3rd element:

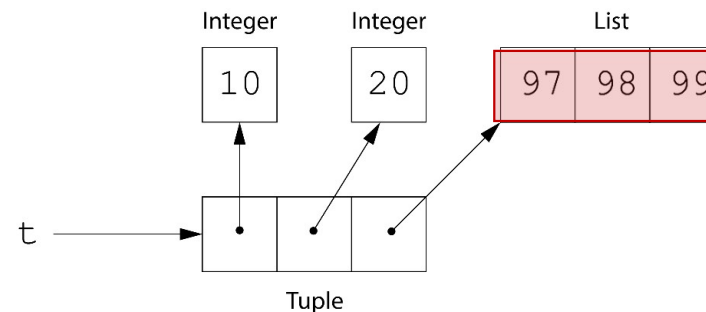
```
t = (10, 20, [97, 98, 99])
```



Storing Mutable Objects in a Tuple

- When a tuple element refers to a mutable object, that object's data can be changed.

```
>>> t = (10, 20, [97, 98, 99])
>>> print(t)
(10, 20, [97, 98, 99])
>>>
>>> t[2].append(100)
>>> print(t)
(10, 20, [97, 98, 99, 100])
>>>
```



The list cannot be changed/replaced as a whole (since the tuple is immutable, but the list items can be modified)

Tuples

Reasons for using tuples:

- a. Processing tuples is faster than processing lists
- b. Tuples provide safety:
 - no elements can be added to or deleted from a tuple
 - once a tuple is created, the elements of the tuple will always refer to the same objects
- c. Some operations in Python require use of tuples

list() function: converts tuple to list

tuple() function: converts list to tuple

Exam Recap

- The following questions were answered correctly by less than 40% of students across all course locations (101 students)
- Let's have a look

Discussion of the hardest exam questions

Q44 (8% answered correctly):

Provide a command to delete key 'k' from dict d.

Accepted solutions:

```
del d['k']  
del d["k"]  
d.pop('k')  
d.pop("k")
```

Discussion of the hardest exam questions

Q43 (53% answered correctly):

What is printed?

```
d = {x: x*x for x in range(3)}  
print(d[2])
```

- a) 0
- b) 1
- c) 2
- d) 4 ✓

Discussion of the hardest exam questions

Q40 (27% answered correctly):

Which dict method safely retrieves a value with a default? (one word)

Accepted solution:

get (upper/lowercase not important)

Discussion of the hardest exam questions

Q34 (50% answered correctly):

Assignment b = a makes b a separate copy of list a.

True or False?

Solution:

False

Discussion of the hardest exam questions

Q29 (58% answered correctly):

What index selects the last element of list a?

Solution:

-1

Discussion of the hardest exam questions

Q25 (31% answered correctly):

What is printed?

```
try:
    x = int('42')
    print('A')
except ValueError:
    print('B')
else:
    print('C')
finally:
    print('D')
```

- a) A C D ✓
- b) A D
- c) B D
- d) C D

Advanced Topics

Type Hints - Motivation

Currently, we considered functions like this:

```
def calculate_discount(price, discount, is_member):  
    if is_member:  
        discount = discount + 0.05  
    return price * (1 - discount)
```

What can go wrong? What would you put for price, discount and is_member?

Type Hints - Motivation

```
def calculate_discount(price, discount, is_member):  
    if is_member:  
        discount = discount + 0.05  
    return price * (1 - discount)
```

What happens?

```
price_item1 = calculate_discount(100, 20, True)  
price_item2 = calculate_discount(100, 0.2, True)  
price_item3 = calculate_discount(100, 20.2, "yes")
```

➔ Example on GitHub

Type Hints – General Structure

For variables:

<variable name>: <type> = <value>

e.g.,

```
x: float = 4.1267
age: int = 39
name: str = "Alice"
price: float = 9.99
is_active: bool = True
```

[Type hints cheat sheet - mypy 1.19.1 documentation](#)

Type Hints – General Structure

For functions:

def <function name>(<variable name>: <type>) -> <type>:

```
def stringify(num: int) -> str:
    return str(num)
```

type of what the
function returns

If a function does not return a value, use None as the return type

Default value for an argument goes after the type annotation

```
def show(value: str, excitement: int = 10) -> None:
    print(value + "!" * excitement)
```

Type Hints – General Structure

For collections on Python 3.9+, the type of the collection item is in brackets

```
list[int] = [1]
set[int] = {6, 7}
```

For mappings, we need the types of both keys and values

```
dict[str, float] = {"field": 2.0} # Python 3.9+
```

For tuples of fixed size, we specify the types of all the elements

```
tuple[int, str, float] = (3, "yes", 7.5) # Python 3.9+
```

For tuples of variable size, we use one type and ellipsis

```
tuple[int, ...] = (1, 2, 3)
```


Type Hints – Multiple Possible and Optional Types

```
from typing import Union, Optional
```

```
# On Python 3.10+, use the | operator when something could be one of a few types
```

```
x: list[int | str] = [3, 5, "test", "fun"] # Python 3.10+
```

```
# On earlier versions, use Union
```

```
x: list[Union[int, str]] = [3, 5, "test", "fun"]
```

```
# Use X | None for a value that could be None on Python 3.10+
```

```
# Use Optional[X] on 3.9 and earlier; Optional[X] is the same as 'X | None'
```

```
x: str | None = "something" if some_condition() else None
```

Type Hints

Type hints allow developers to annotate code with expected types for variables and function arguments.

- Helps improve code readability
- «Self-documenting» code
- Catch errors before runtime
- Convenient when using IDEs to get suggestions and tools like mypy



Type hints are not enforced by the interpreter!

Type Hints – Type Alias

For improved readability, we can also create an alias for type hints, e.g.,

```
StudentID = int
GradeBook = dict[StudentID, list[float]]

def average_grade(grades: GradeBook, student: StudentID) -> float:
    return sum(grades[student]) / len(grades[student])
```

From Python 3.12+, you can also use the type statement

```
type StudentID = int
```

Enumerators

When iterating through, e.g., a list, how do you get the index?

For example like this:

```
products = ["Laptop", "Mouse", "GPU", "Flux Capacitor"]
```

```
i = 0
```

```
for product in products:
```

```
    print(f"{i}: {product}")
```

```
    i += 1
```

Enumerators

There is a better more pythonic way:

```
products = ["Laptop", "Mouse", "GPU", "Flux Capacitor"]
```

```
for idx, product in enumerate(products):  
    print(f"{idx}: {product}")
```

➔ `enumerate()` gives you an (index, item) tuple

Enumerators – Basic Usage

any (valid) variable name you like

```
for i, char in enumerate("ABC"):
    print(i, char)
```

```
# 0 A
# 1 B
# 2 C
```

```
winners = ["Alice", "Bob", "Carol"]
for rank, name in enumerate(winners, start=1):
    print(f"#{rank}: {name}")
```

```
# #1: Alice
# #2: Bob
# #3: Carol
```

Custom start index

Generator Functions

Generators are special functions that returns an iterator object (an object that contains a countable number of values, we can traverse through all values)

Generators have many desirable properties:

- **Memory efficient:** Processes large data streams without storing everything in memory.
- **No list overhead:** Produces values one at a time instead of creating a full list upfront.
- **Lazy evaluation:** Computes results only when they are actually needed.
- **Handles infinite sequences:** Suited for unbounded sequences
- **Pipeline-friendly:** Can be combined in stages to build efficient data processing flows.

Generators: Basics

When you need to process large lists, you probably would have written something like this:

```
def get_squares(n):  
    result = []  
    for x in range(n):  
        result.append(x ** 2)  
    return result
```

If you want to, e.g., compute `get_squares(10000000)`, you have to load and wait for the entire list

Generators: Basics

With generators, you can process on demand:

```
def square_counter(max):  
    count = 1  
    while count <= max:  
        yield count ** 2  
        count += 1
```

```
# test with  
ctr = square_counter(5)  
for n in ctr:  
    print(n)
```

You can also generate the next item with
`next(generator)`, e.g.,

```
ctr = square_counter(5)  
next(ctr)
```

Generators: Yield

You can define a generator like a function with at least one yield statement (while regular functions use return statement).

```
def generator_function_name(parameters):  
    # Your code here  
    yield expression  
    # Additional code can follow
```

Return: Exits function and returns final value, function state is lost.

Yield: Pauses function, saves state, allows continuing from the same point. Computes on demand

Generators – Example

Generator comprehensions: Very similar to list comprehensions

```
squares = (x**2 for x in range(1000))
```

Example:

```
def counter():  
    """Generator keeps its state between yields."""  
    count = 0  
    while True: # Infinite generator!  
        yield count  
        count += 1
```

When use generators, and when lists?

Use generators, when:

- Data does not fit into memory (e.g., very large CSV files)
- Streaming data and data pipelines
- you might not need all data items
- representing infinite sequences

Use lists, when:

- you want to iterate multiple times
- you need the list length at the beginning
- you need to modify item
- reversing or slicing is needed
- you need random access (item[N])
- data fits in memory



Conversion from generators to list: `list(generator)`, use only when needed!

E.g. `list(range(5))` gives `[0, 1, 2, 3, 4]`

Procedure Models

What Are Procedure Models?

- Structured approaches to organizing and managing software development
- Define phases, roles, responsibilities, and workflows
- Help teams deliver software systematically and predictably
- Provide guidance for:
 - Planning
 - Development
 - Testing
 - Deployment
 - Maintenance
- Reduce risk through clear processes and transparency
- Important in business environments for cost control and quality assurance

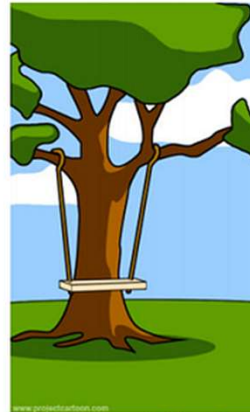
How Projects Really Work (version 1.0)

Create your own cartoon at www.projectcartoon.com

How To Avoid This



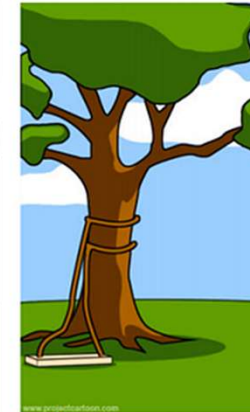
How the customer explained it



How the project leader understood it



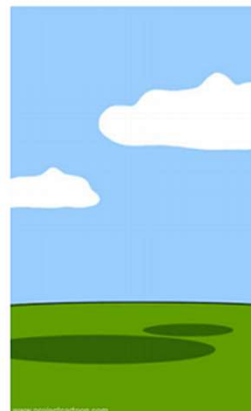
How the analyst designed it



How the programmer wrote it



How the business consultant described it



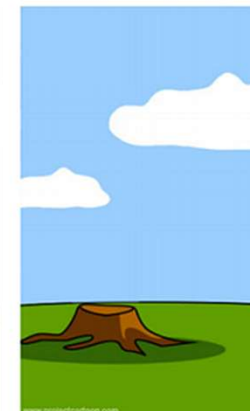
How the project was documented



What operations installed



How the customer was billed

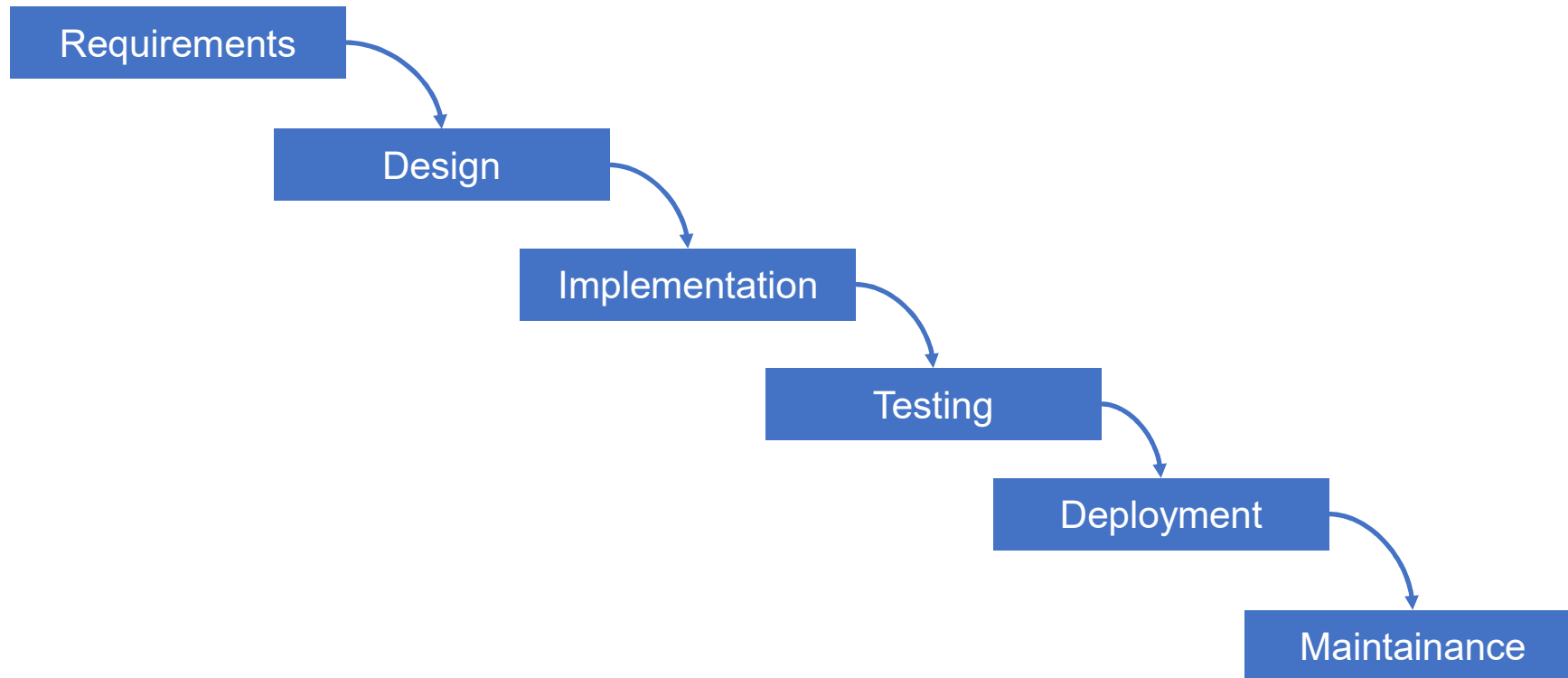


How it was supported



What the customer really needed

Waterfall Model



Waterfall Model – Characteristics

- Sequential / Linear Progression
Each stage must be completed before moving on
- Frozen Requirements
Requirements are fixed and not revised
- Documentation
Extensive documentation upfront and at each stage
- Late Testing
Testing at the end of the implementation, near the end of the project
- Deployment
Product is delivered after full completion

Waterfall Model – Pros & Cons

Advantages

- ✓ Clear milestones & deliverables
- ✓ Easy to plan and budget
- ✓ Well-documented output
- ✓ Simple to understand

Disadvantages

- ⚠ Late feedback from users
- ⚠ High cost of changes
- ⚠ Risk of building wrong thing
- ⚠ No working software until end

Good for projects in regulated industries or high-risk applications (medical, aviation, military, ...), well-understood domains

Scrum

Scrum is an agile framework for developing, delivering, and sustaining complex products through iterative and incremental practices.



Sprint

Fixed time-box (~1-4 weeks). No changes that endanger the Sprint Goal; Quality
The Product Backlog is refined, Scope clarified/renegotiated with the Product Owner if needed.



Sprint Backlog

Committed work for sprint. ordered list of what is needed to improve the product. It is the single source of work undertaken by the Scrum Team.



Product Backlog

Prioritized feature list.



Increment

Shippable product each sprint

Scrum – Events („Ceremonies“)



Sprint Planning

What will we deliver? How will we do it?

Typical duration

2-4 h



Daily Standup

What did I do? What will I do? Any blockers?

15min



Sprint Review

Demo to stakeholders, gather feedback

1-2 h



Retrospective

What went well? What to improve?

1-2 h

Scrum Roles



Product Owner

- Defines the product vision
- Prioritizes the backlog
- Represents stakeholders
- Accepts/rejects work



Scrum Master

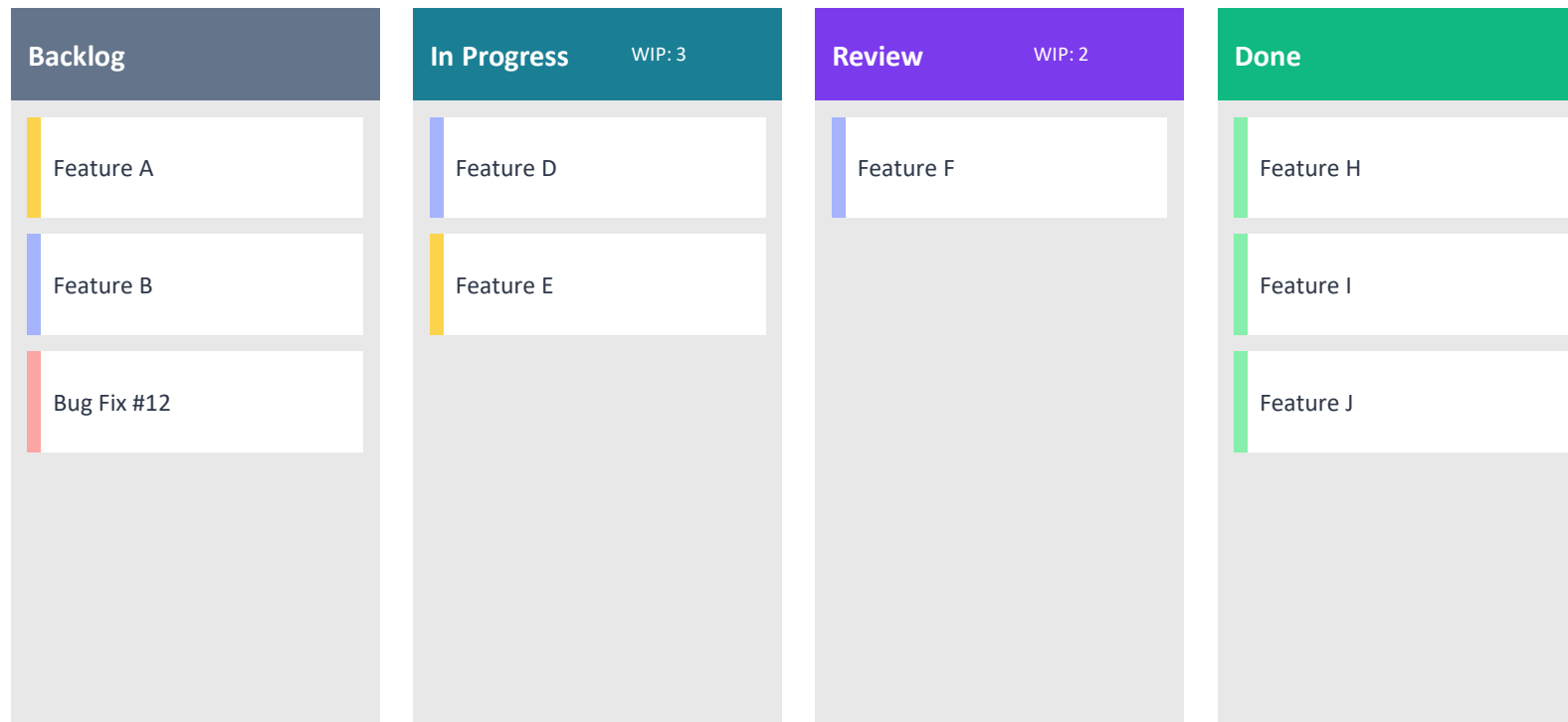
- Facilitates Ceremonies
- Removes impediments
- Team coach
- Protects team from distractions



Development Team

- Delivers increments
- Self-organizing
- Cross-functional
- 10 or fewer people

Kanban – The Board



Kanban – Principles

1. Visualize workflows: Kanban boards visualize workflow using columns like **“To Do,” “In Progress,” and “Done.”** Each card in a column represents a task or user story and contains key details such as deadlines and responsible team members.
2. Limit work-in-progress: Multitasking reduces productivity, so Kanban limits **work in progress (WIP)**. By setting WIP limits per column, teams focus on finishing tasks instead of starting too many at once. The goal is steady flow without multitasking
3. Manage the flow: The board shows bottlenecks
4. Transparent policies: What is needed to move a card forward?
5. Feedback loops
6. Collaborative Improvement

User Stories

- Should effectively capture requirements
- Smallest unit of work in an agile framework. It's an end goal, not a feature, expressed from the software user's perspective.
- In scrum, user stories are added to sprints and “burned down” over the duration of the sprint.
- Format
As a **[role]**, I want **[feature]** so that **[benefit]**.
- Including **acceptance criteria** helps teams understand what success looks like and ensures that the story is testable.

Criteria for Good User Stories (INVEST)

Independent – can be developed separately

Negotiable – details can be discussed

Valuable – provides business value

Estimable – effort can be estimated

Small – fits into one iteration

Testable – acceptance criteria possible

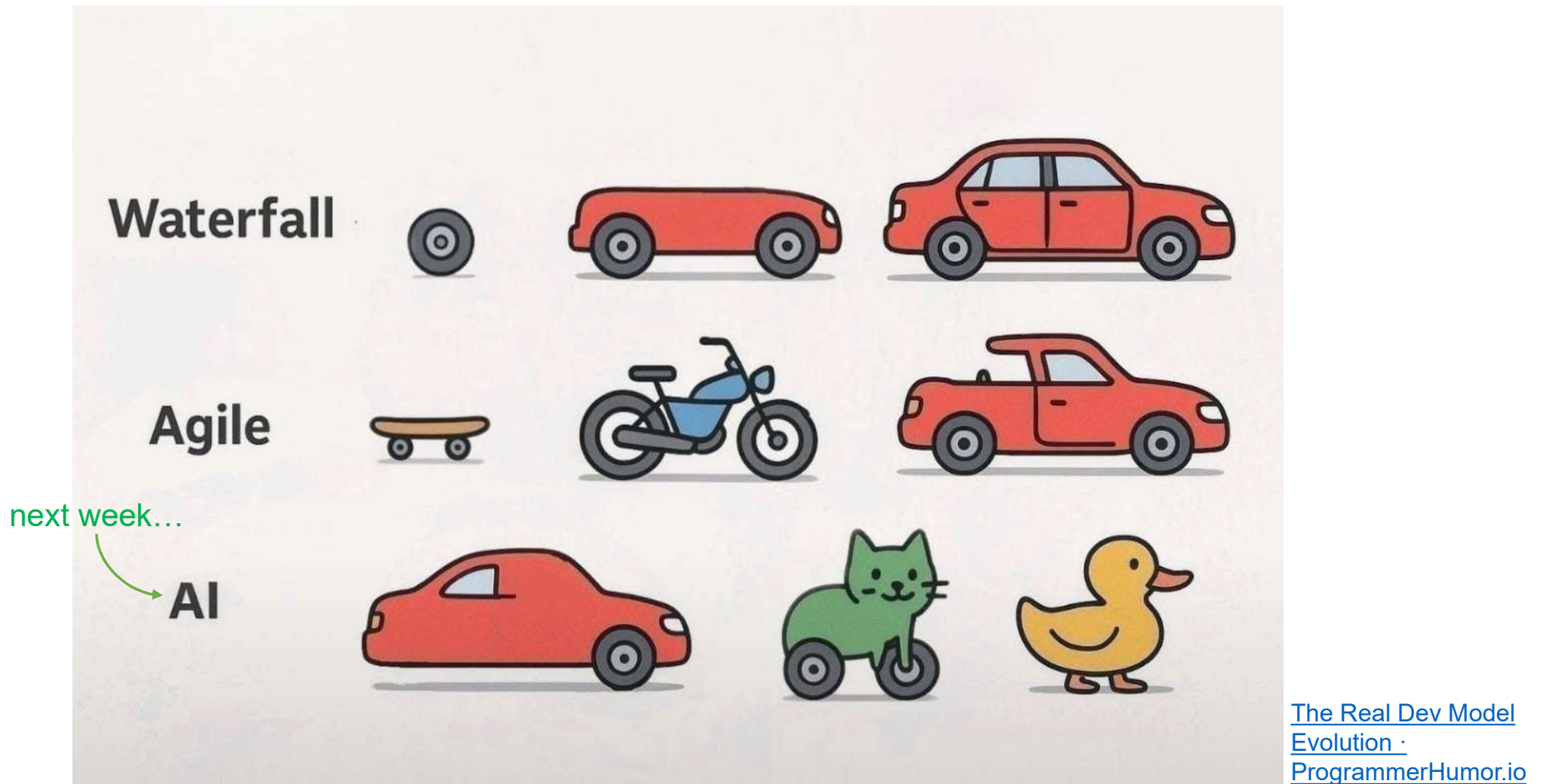
Acceptance Criteria for User Stories

User story: As a customer, I want to search for products by name to find certain items easily.

Acceptance criteria:

- The search function should return results that exactly match the entered product name.
- The search function should also return results that partially match the entered product name (with at least three matching characters).
- Search results should be displayed clearly and organized, including the product name, image, and price.
- The search results page should allow pagination to display a maximum of 20 items per page.

example by Atlassian (<https://www.atlassian.com/work-management/project-management/acceptance-criteria>)



Exercise

Discuss in the project groups from last semester. Write a user story including acceptance criteria for your previous semester project.